

Today

Assignment 1

- System calls in OS/161

Week 2 Questions

- Finish off critical sections

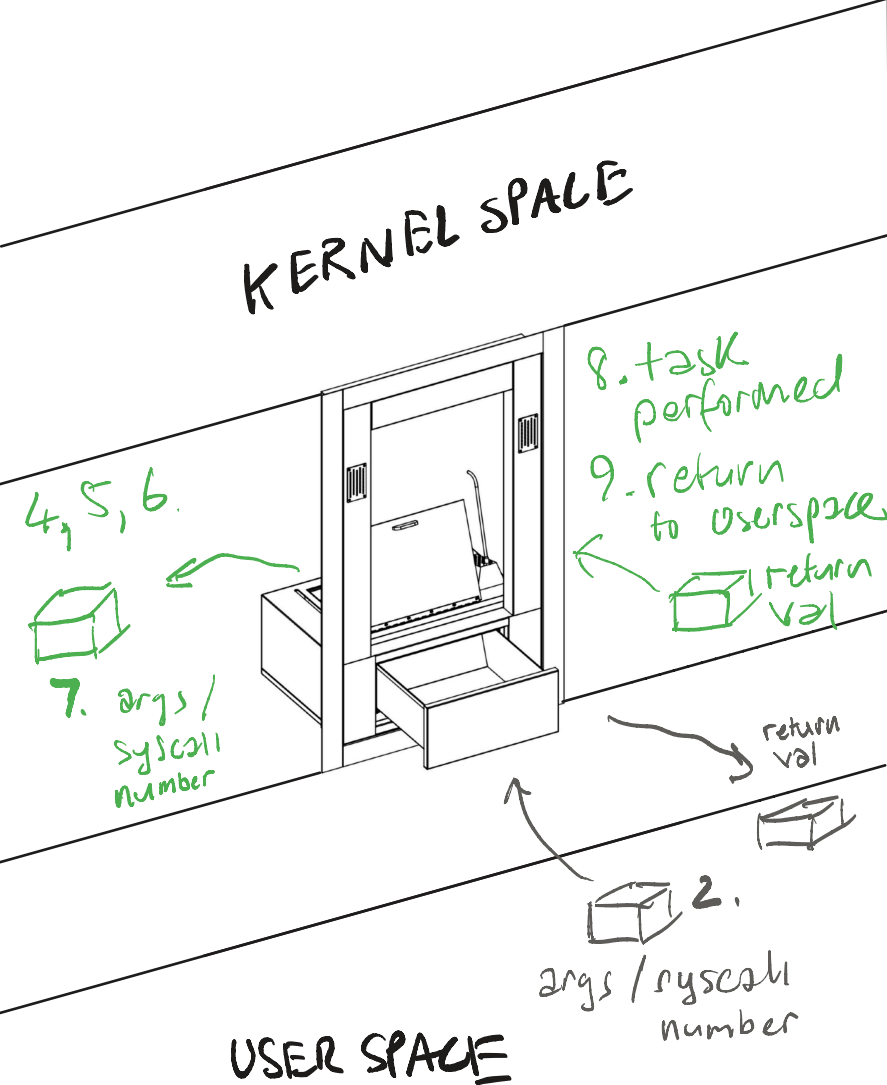
Synchronisation Primitives

- Locks
- Semaphores
- Condition variables

Week 3 Questions

- 1–4 (time allowing)

KERNEL SPACE



USER SPACE

1. user program calls library function (e.g. open)
2. library function is backed by assembly routine, which places syscall number and arguments in registers
3. syscall executed; control transferred to operating system via exception
4. exception type checked
5. syscall exception identified
6. syscall handler executed
7. syscall identified by number; args checked

Critical sections

14. The following fragment of code is a single line of C code. How might a race condition occur if it's executed concurrently in multiple threads? Can you give an example of how an incorrect result can be computed for x ?

$x = x + 1;$ (assume x is global)

↑ Statement is not atomic

Thread 1:

lh t1 x



Context
switches



addi t1 1

sh t1 x

Thread 2:

lh t1 x

addi t1 1

sh t1 x

Thread 1 has
a stale copy

of x , so x

is only incremented

by 1, rather

than 2.

15. The following function is called by multiple threads (potentially concurrently) in a multi-threaded program. Identify the critical section(s) that require(s) mutual exclusion. Describe the race condition or why no race condition exists.

```
int i;  
  
void foo()  
{  
    int j;  
  
    /* random stuff */  
  
    i = i + 1;  
    j = j + 1;  
  
    /* more random stuff */  
}
```

*i is global
potential for race*

j is local; no race

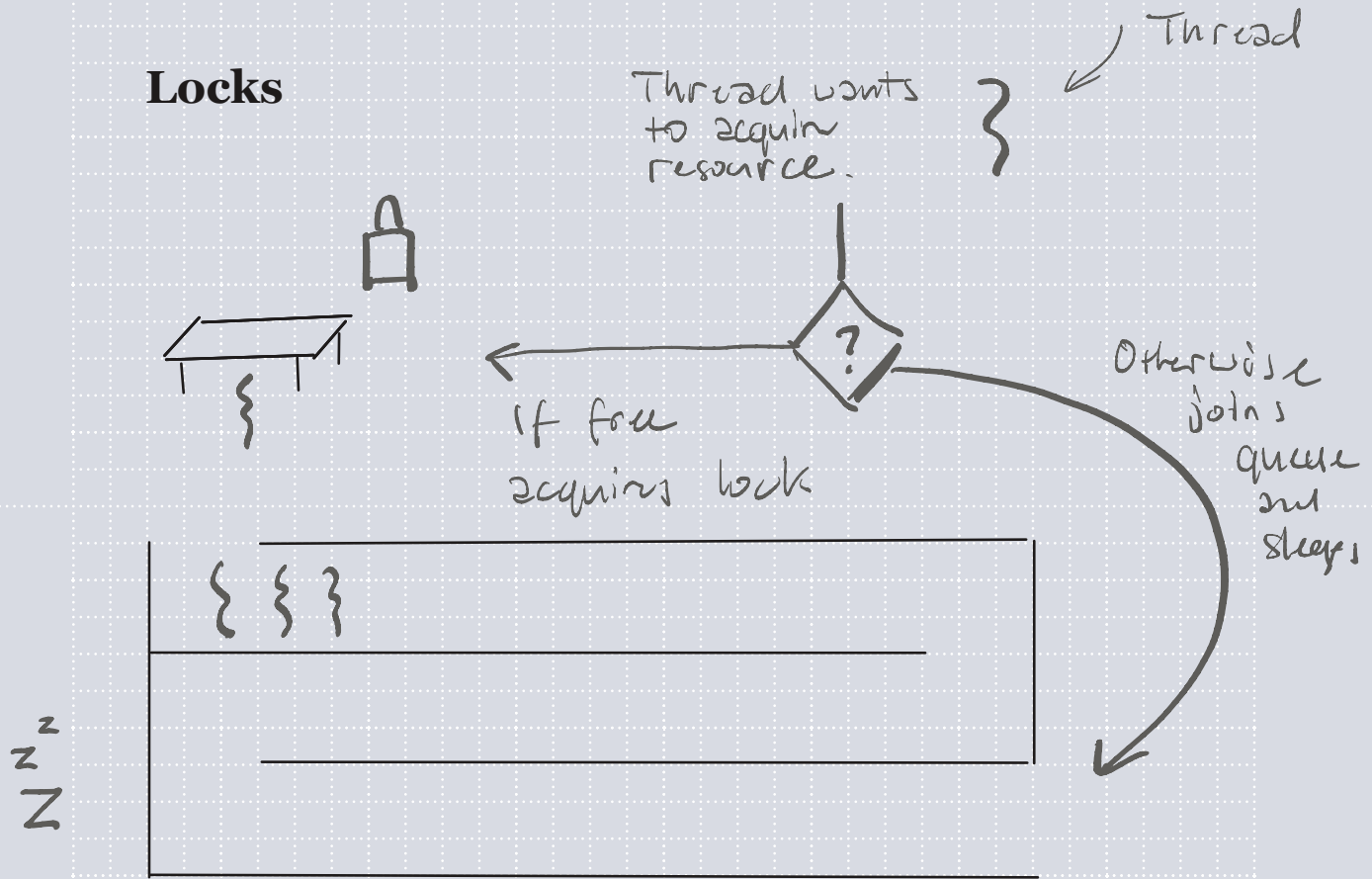
16. The following function is called by threads in a multi-thread program. Under what conditions would it form a critical section.

```
void inc_mem(int *iptr)
{
    *iptr = *iptr + 1;
}
```

Critical section if argument is global.

(context: Queuing at Nucleus)

Locks



Releasing the lock will wake one thread from the queue, allowing it to take the lock

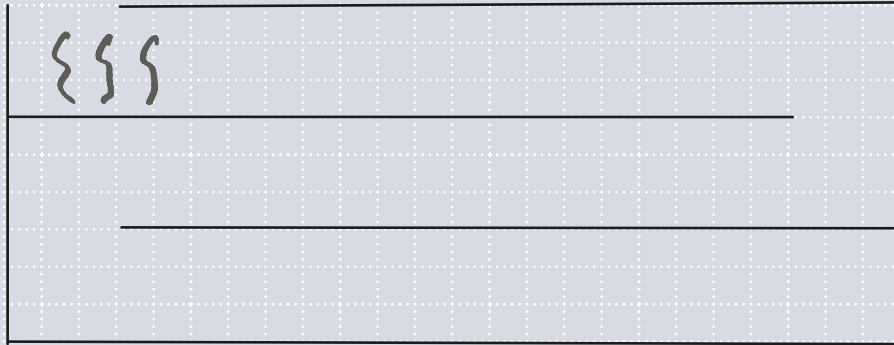
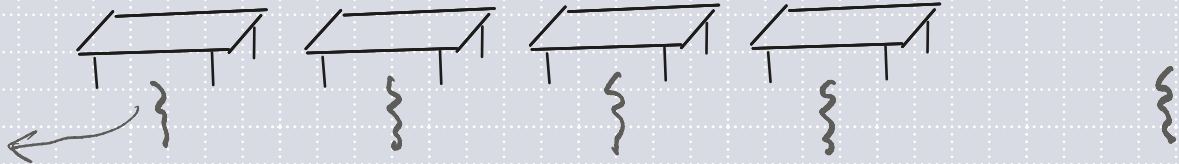
Semaphores

$\text{sem desks} = 4$ (initially)

2.

When thread leaves a desk, calls $V(\text{desks})$, which increments desks and wakes a sleeping thread

z
z
Z



1. Thread calls $P(\text{desks})$ on entry.

If desks is 0, thread joins wait queue.

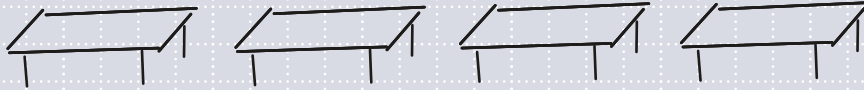
P (also known as down or wait)

V (also known as up or signal)

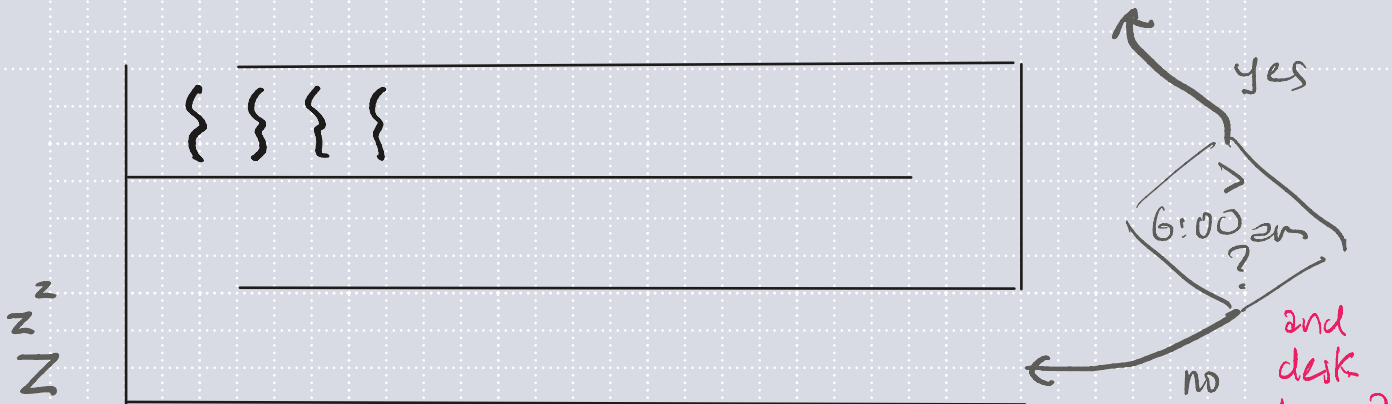
Otherwise, immediately accesses desks.

(decrements desks)

Condition Variables



If time later than 6:00 am,
allow entry



Conditional variables are even more
general!

Coordinating activities

1. What synchronisation mechanism or approach might one take to have one thread wait for another thread to update some state?

sem update = 0;

Thread 1:
P(update)

Thread 2:
V(update)

1. Thread 1 sleeps immediately update = 0

2. Thread 2 V() signals Thread 1 update = 1

3. Thread 1 wakes up update = 0

2. A particular abstraction only allows a maximum of 10 threads to enter the "room" at any point in time. Further threads attempting to enter the room have to wait at the door for another thread to exit the room. How could one implement a synchronisation approach to enforce the above restriction?

- A semaphore initialised to 10 would be a natural approach.
- A condition variable could work too

3. Multiple threads are waiting for the same thing to happen (e.g. a disk block to arrive from disk). Write pseudo-code for synchronising and waking the multiple threads waiting for the same event.

```
wait_block() {  
    acquire_lock(lock);  
    while(status == BLOCK_NOT_READY){  
        cv_wait(cv, lock);  
    }  
    release_lock(lock);  
}
```

```
block_ready() {  
    acquire_lock(lock);  
    status = BLOCK_READY;  
    cv_broadcast(cv, lock);  
    release_lock(lock);  
}
```

Identify Deadlocks

4. Here are code samples from two threads that use semaphores (count initialised to 1). Give a sequence of execution and context switches in which these two threads can deadlock.

Propose a change to one or both of them that makes deadlock impossible. What general principle do the original threads violate that causes them to deadlock.

```
semaphore *mutex, *data;
```

```
void me() {  
  1 P(mutex);  
    /* do something */  
  
  4 P(data);  
    /* do something else */  
  
    V(mutex);  
  
    /* clean up */  
    V(data);  
}
```

```
void you() {  
  2 P(data);  
  3 P(mutex);  
  
    /* do something */  
  
    V(data);  
    V(mutex);  
}
```

P (down or wait) ; V (up or signal)

0. initial state

mutex = 1, data = 1

1. me(): P(mutex)

mutex = 0, data = 1

- Context switch

2. you(): P(data)

mutex = 0, data = 0

3. you(): P(mutex)

you thread sleeps

- context switch

4. me(): P(data)

me thread sleeps

- deadlock

Threads don't follow global lock ordering

me():

P(mutex)

P(data)

V(data)

V(mutex)

would

not

deadlock

you():

P(mutex)

P(data)

V(data)

V(mutex)